



# Complete MERN Stack Project: Task Manager

A full-stack task management application demonstrating all MERN concepts.

---



## Project Structure

```
task-manager/
├── backend/
│   ├── config/
│   │   └── db.js
│   ├── controllers/
│   │   ├── authController.js
│   │   └── taskController.js
│   ├── middleware/
│   │   ├── authMiddleware.js
│   │   └── errorMiddleware.js
│   ├── models/
│   │   ├── User.js
│   │   └── Task.js
│   ├── routes/
│   │   ├── authRoutes.js
│   │   └── taskRoutes.js
│   ├── .env
│   ├── package.json
│   └── server.js
├── frontend/
│   ├── src/
│   │   ├── components/
│   │   │   ├── Header.jsx
│   │   │   ├── TaskForm.jsx
│   │   │   ├── TaskItem.jsx
│   │   │   └── TaskList.jsx
│   │   ├── context/
│   │   │   └── AuthContext.jsx
│   │   ├── hooks/
│   │   │   └── useFetch.js
│   │   ├── pages/
│   │   │   ├── Home.jsx
│   │   │   ├── Login.jsx
│   │   │   ├── Register.jsx
│   │   │   └── Dashboard.jsx
│   │   ├── App.jsx
│   │   ├── index.css
│   │   └── main.jsx
│   └── package.json
└── README.md
```

# BACKEND CODE

## 1. package.json (Backend)

```
{
  "name": "task-manager-backend",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "bcryptjs": "^2.4.3",
    "cors": "^2.8.5",
    "dotenv": "^16.0.0",
    "express": "^4.18.2",
    "jsonwebtoken": "^9.0.0",
    "mongoose": "^7.0.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.0"
  }
}
```

## 2. .env (Backend)

```
PORT=5000
MONGODB_URI=mongodb://localhost:27017/taskmanager
JWT_SECRET=your_super_secret_key_change_this_in_production
JWT_EXPIRE=7d
```

## 3. config/db.js - Database Connection

```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGODB_URI);
    console.log(`✅ MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`❌ Error: ${error.message}`);
    process.exit(1);
  }
};
```

```
module.exports = connectDB;
```

## 4. models/User.js - User Schema

```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please add a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please add an email'],
    unique: true,
    lowercase: true,
    match: [/^\S+@\S+\.\S+$/, 'Please add a valid email']
  },
  password: {
    type: String,
    required: [true, 'Please add a password'],
    minlength: [6, 'Password must be at least 6 characters'],
    select: false // Don't return password in queries
  }
}, {
  timestamps: true
});

// Hash password before saving
userSchema.pre('save', async function(next) {
  // Only hash if password is modified
  if (!this.isModified('password')) {
    return next();
  }

  // Hash password with bcrypt
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Method to compare passwords
userSchema.methods.matchPassword = async function(enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};
```

```
module.exports = mongoose.model('User', userSchema);
```

---

## 5. models/Task.js - Task Schema

```
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema({
  title: {
    type: String,
    required: [true, 'Please add a task title'],
    trim: true,
    maxlength: [100, 'Title cannot be more than 100 characters']
  },
  description: {
    type: String,
    maxlength: [500, 'Description cannot be more than 500 characters']
  },
  status: {
    type: String,
    enum: ['pending', 'in-progress', 'completed'],
    default: 'pending'
  },
  priority: {
    type: String,
    enum: ['low', 'medium', 'high'],
    default: 'medium'
  },
  dueDate: {
    type: Date
  },
  // Reference to User who created this task
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  }
}, {
  timestamps: true
});

module.exports = mongoose.model('Task', taskSchema);
```

---

## 6. middleware/authMiddleware.js - JWT Authentication

```
const jwt = require('jsonwebtoken');
const User = require('../models/User');
```

```

const protect = async (req, res, next) => {
  let token;

  // Check for token in Authorization header
  if (req.headers.authorization &&
    req.headers.authorization.startsWith('Bearer')) {

    try {
      // Get token from header (Bearer TOKEN)
      token = req.headers.authorization.split(' ')[1];

      // Verify token
      const decoded = jwt.verify(token, process.env.JWT_SECRET);

      // Get user from token (exclude password)
      req.user = await User.findById(decoded.id).select('-password');

      if (!req.user) {
        return res.status(401).json({ message: 'User not found' });
      }

      next();
    } catch (error) {
      console.error(error);
      res.status(401).json({ message: 'Not authorized, token failed' });
    }
  }

  if (!token) {
    res.status(401).json({ message: 'Not authorized, no token' });
  }
};

module.exports = { protect };

```

## 7. middleware/errorMiddleware.js - Error Handler

```

const errorHandler = (err, req, res, next) => {
  let statusCode = res.statusCode === 200 ? 500 : res.statusCode;
  let message = err.message;

  // Mongoose bad ObjectId
  if (err.name === 'CastError' && err.kind === 'ObjectId') {
    statusCode = 404;
    message = 'Resource not found';
  }

  // Mongoose duplicate key
  if (err.code === 11000) {
    statusCode = 400;
  }

```

```

        message = 'Duplicate field value entered';
    }

    // Mongoose validation error
    if (err.name === 'ValidationError') {
        statusCode = 400;
        message = Object.values(err.errors).map(e => e.message).join(', ');
    }

    res.status(statusCode).json({
        message,
        stack: process.env.NODE_ENV === 'production' ? null : err.stack
    });
};

module.exports = { errorHandler };

```

## 8. controllers/authController.js - Auth Logic

```

const jwt = require('jsonwebtoken');
const User = require('../models/User');

// Generate JWT Token
const generateToken = (id) => {
    return jwt.sign({ id }, process.env.JWT_SECRET, {
        expiresIn: process.env.JWT_EXPIRE
    });
};

// @desc    Register new user
// @route   POST /api/auth/register
// @access  Public
const register = async (req, res) => {
    try {
        const { name, email, password } = req.body;

        // Check if user exists
        const userExists = await User.findOne({ email });
        if (userExists) {
            res.status(400);
            throw new Error('User already exists');
        }

        // Create user
        const user = await User.create({
            name,
            email,
            password
        });
    }
};

```

```

    if (user) {
      res.status(201).json({
        _id: user._id,
        name: user.name,
        email: user.email,
        token: generateToken(user._id)
      });
    }
  } catch (error) {
    res.status(res.statusCode === 200 ? 500 : res.statusCode);
    throw error;
  }
};

// @desc    Login user
// @route    POST /api/auth/login
// @access   Public
const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    // Check for user email (include password for comparison)
    const user = await User.findOne({ email }).select('+password');

    if (user && (await user.matchPassword(password))) {
      res.json({
        _id: user._id,
        name: user.name,
        email: user.email,
        token: generateToken(user._id)
      });
    } else {
      res.status(401);
      throw new Error('Invalid email or password');
    }
  } catch (error) {
    res.status(res.statusCode === 200 ? 500 : res.statusCode);
    throw error;
  }
};

// @desc    Get current user
// @route    GET /api/auth/me
// @access   Private
const getMe = async (req, res) => {
  res.json(req.user);
};

module.exports = { register, login, getMe };

```

## 9. controllers/taskController.js - Task CRUD

```
const Task = require('../models/Task');

// @desc    Get all tasks for logged in user
// @route    GET /api/tasks
// @access   Private
const getTasks = async (req, res) => {
  try {
    const tasks = await Task.find({ user: req.user._id })
      .sort({ createdAt: -1 });
    res.json(tasks);
  } catch (error) {
    res.status(500);
    throw error;
  }
};

// @desc    Get single task
// @route    GET /api/tasks/:id
// @access   Private
const getTask = async (req, res) => {
  try {
    const task = await Task.findById(req.params.id);

    if (!task) {
      res.status(404);
      throw new Error('Task not found');
    }

    // Check that task belongs to user
    if (task.user.toString() !== req.user._id.toString()) {
      res.status(401);
      throw new Error('Not authorized');
    }

    res.json(task);
  } catch (error) {
    res.status(res.statusCode === 200 ? 500 : res.statusCode);
    throw error;
  }
};

// @desc    Create task
// @route    POST /api/tasks
// @access   Private
const createTask = async (req, res) => {
  try {
    const { title, description, status, priority, dueDate } = req.body;
```

```

    const task = await Task.create({
      title,
      description,
      status,
      priority,
      dueDate,
      user: req.user._id // Set user from auth middleware
    });

    res.status(201).json(task);
  } catch (error) {
    res.status(400);
    throw error;
  }
};

// @desc    Update task
// @route    PUT /api/tasks/:id
// @access   Private
const updateTask = async (req, res) => {
  try {
    let task = await Task.findById(req.params.id);

    if (!task) {
      res.status(404);
      throw new Error('Task not found');
    }

    // Check ownership
    if (task.user.toString() !== req.user._id.toString()) {
      res.status(401);
      throw new Error('Not authorized');
    }

    task = await Task.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );

    res.json(task);
  } catch (error) {
    res.status(res.statusCode === 200 ? 500 : res.statusCode);
    throw error;
  }
};

// @desc    Delete task
// @route    DELETE /api/tasks/:id
// @access   Private
const deleteTask = async (req, res) => {
  try {

```

```

const task = await Task.findById(req.params.id);

if (!task) {
  res.status(404);
  throw new Error('Task not found');
}

// Check ownership
if (task.user.toString() !== req.user._id.toString()) {
  res.status(401);
  throw new Error('Not authorized');
}

await Task.findByIdAndDelete(req.params.id);

res.json({ message: 'Task deleted' });
} catch (error) {
  res.status(res.statusCode === 200 ? 500 : res.statusCode);
  throw error;
}
};

module.exports = { getTasks, getTask, createTask, updateTask, deleteTask };

```

## 10. routes/authRoutes.js

```

const express = require('express');
const router = express.Router();
const { register, login, getMe } = require('../controllers/authController');
const { protect } = require('../middleware/authMiddleware');

router.post('/register', register);
router.post('/login', login);
router.get('/me', protect, getMe);

module.exports = router;

```

## 11. routes/taskRoutes.js

```

const express = require('express');
const router = express.Router();
const {
  getTasks,
  getTask,
  createTask,
  updateTask,
  deleteTask
} = require('../controllers/taskController');

```

```
const { protect } = require('../middleware/authMiddleware');

// All routes need authentication
router.use(protect);

router.route('/')
  .get(getTasks)
  .post(createTask);

router.route('/:id')
  .get(getTask)
  .put(updateTask)
  .delete(deleteTask);

module.exports = router;
```

---

## 12. server.js - Main Entry Point

```
const express = require('express');
const cors = require('cors');
const dotenv = require('dotenv');
const connectDB = require('./config/db');
const { errorHandler } = require('../middleware/errorMiddleware');

// Load env vars
dotenv.config();

// Connect to database
connectDB();

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// Routes
app.use('/api/auth', require('./routes/authRoutes'));
app.use('/api/tasks', require('./routes/taskRoutes'));

// Health check route
app.get('/api/health', (req, res) => {
  res.json({ status: 'OK', message: 'Server is running' });
});

// Error handler (must be last)
app.use(errorHandler);

// Start server
const PORT = process.env.PORT || 5000;
```

```
app.listen(PORT, () => {
  console.log(`🚀 Server running on port ${PORT}`);
});
```

---

## FRONTEND CODE

### 13. package.json (Frontend - Vite)

```
{
  "name": "task-manager-frontend",
  "version": "1.0.0",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^18.2.0",
    "react-dom": "^18.2.0",
    "react-router-dom": "^6.0.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "^4.0.0",
    "vite": "^5.0.0"
  }
}
```

---

### 14. src/context/AuthContext.jsx - Global Auth State

```
import { createContext, useContext, useState, useEffect } from 'react';

const AuthContext = createContext();

export function AuthProvider({ children }) {
  const [user, setUser] = useState(null);
  const [loading, setLoading] = useState(true);

  // Check for saved user on mount
  useEffect(() => {
    const savedUser = localStorage.getItem('user');
    if (savedUser) {
      setUser(JSON.parse(savedUser));
    }
    setLoading(false);
  }, []);

  // Login function
```

```

const login = async (email, password) => {
  const response = await fetch('http://localhost:5000/api/auth/login', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ email, password })
  });

  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message || 'Login failed');
  }

  localStorage.setItem('user', JSON.stringify(data));
  setUser(data);
  return data;
};

// Register function
const register = async (name, email, password) => {
  const response = await fetch('http://localhost:5000/api/auth/register', {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ name, email, password })
  });

  const data = await response.json();

  if (!response.ok) {
    throw new Error(data.message || 'Registration failed');
  }

  localStorage.setItem('user', JSON.stringify(data));
  setUser(data);
  return data;
};

// Logout function
const logout = () => {
  localStorage.removeItem('user');
  setUser(null);
};

const value = {
  user,
  loading,
  login,
  register,
  logout,
  isAuthenticated: !!user
};

```

```

    return (
      <AuthContext.Provider value={value}>
        {!loading && children}
      </AuthContext.Provider>
    );
  }

export function useAuth() {
  const context = useContext(AuthContext);
  if (!context) {
    throw new Error('useAuth must be used within AuthProvider');
  }
  return context;
}

```

## 15. src/hooks/useFetch.js - Custom Fetch Hook

```

import { useState, useEffect } from 'react';
import { useAuth } from '../context/AuthContext';

export function useFetch(url) {
  const [data, setData] = useState(null);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState(null);
  const { user } = useAuth();

  useEffect(() => {
    const fetchData = async () => {
      try {
        setLoading(true);
        const response = await fetch(`http://localhost:5000${url}`, {
          headers: {
            'Authorization': `Bearer ${user?.token}`
          }
        });

        const result = await response.json();

        if (!response.ok) {
          throw new Error(result.message);
        }

        setData(result);
      } catch (err) {
        setError(err.message);
      } finally {
        setLoading(false);
      }
    };
  });
}

```

```

        if (user) {
            fetchData();
        }
    }, [url, user]);

    return { data, loading, error, setData };
}

```

## 16. src/components/Header.jsx

```

import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export default function Header() {
    const { user, logout, isAuthenticated } = useAuth();
    const navigate = useNavigate();

    const handleLogout = () => {
        logout();
        navigate('/login');
    };

    return (
        <header className="header">
            <div className="container">
                <Link to="/" className="logo"> 📄 Task Manager </Link>

                <nav>
                    {isAuthenticated ? (
                        <>
                            <span>Hello, {user.name}</span>
                            <Link to="/dashboard">Dashboard</Link>
                            <button onClick={handleLogout}>Logout</button>
                        </>
                    ) : (
                        <>
                            <Link to="/login">Login</Link>
                            <Link to="/register">Register</Link>
                        </>
                    )}
                </nav>
            </div>
        </header>
    );
}

```

## 17. src/components/TaskForm.jsx

```
import { useState } from 'react';
import { useAuth } from '../context/AuthContext';

export default function TaskForm({ onTaskAdded }) {
  const [title, setTitle] = useState('');
  const [description, setDescription] = useState('');
  const [priority, setPriority] = useState('medium');
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState('');
  const { user } = useAuth();

  const handleSubmit = async (e) => {
    e.preventDefault();

    if (!title.trim()) {
      setError('Title is required');
      return;
    }

    try {
      setLoading(true);
      setError('');

      const response = await fetch('http://localhost:5000/api/tasks', {
        method: 'POST',
        headers: {
          'Content-Type': 'application/json',
          'Authorization': `Bearer ${user.token}`
        },
        body: JSON.stringify({ title, description, priority })
      });

      const data = await response.json();

      if (!response.ok) {
        throw new Error(data.message);
      }

      // Clear form
      setTitle('');
      setDescription('');
      setPriority('medium');

      // Notify parent
      onTaskAdded(data);
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  }
}
```

```

    });

    return (
      <form onSubmit={handleSubmit} className="task-form">
        <h3>Add New Task</h3>

        {error && <div className="error">{error}</div>}

        <div className="form-group">
          <input
            type="text"
            placeholder="Task title"
            value={title}
            onChange={(e) => setTitle(e.target.value)}
          />
        </div>

        <div className="form-group">
          <textarea
            placeholder="Description (optional)"
            value={description}
            onChange={(e) => setDescription(e.target.value)}
            rows={3}
          />
        </div>

        <div className="form-group">
          <select
            value={priority}
            onChange={(e) => setPriority(e.target.value)}
          >
            <option value="low">Low Priority</option>
            <option value="medium">Medium Priority</option>
            <option value="high">High Priority</option>
          </select>
        </div>

        <button type="submit" disabled={loading}>
          {loading ? 'Adding...' : 'Add Task'}
        </button>
      </form>
    );
  }
}

```

## 18. src/components/TaskItem.jsx

```

import { useState } from 'react';
import { useAuth } from '../context/AuthContext';

export default function TaskItem({ task, onUpdate, onDelete }) {

```

```

const [isEditing, setIsEditing] = useState(false);
const [title, setTitle] = useState(task.title);
const [status, setStatus] = useState(task.status);
const { user } = useAuth();

const handleStatusChange = async (newStatus) => {
  try {
    const response = await fetch(`http://localhost:5000/api/tasks/${task._id}`, {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
        'Authorization': `Bearer ${user.token}`
      },
      body: JSON.stringify({ status: newStatus })
    });

    const data = await response.json();

    if (response.ok) {
      setStatus(newStatus);
      onUpdate(data);
    }
  } catch (err) {
    console.error(err);
  }
};

const handleDelete = async () => {
  if (!window.confirm('Delete this task?')) return;

  try {
    const response = await fetch(`http://localhost:5000/api/tasks/${task._id}`, {
      method: 'DELETE',
      headers: {
        'Authorization': `Bearer ${user.token}`
      }
    });

    if (response.ok) {
      onDelete(task._id);
    }
  } catch (err) {
    console.error(err);
  }
};

const priorityColors = {
  low: '#10b981',
  medium: '#f59e0b',
  high: '#ef4444'
};

```

```

return (
  <div className={`task-item ${status}`}>
    <div className="task-content">
      <span
        className="priority-badge"
        style={{ backgroundColor: priorityColors[task.priority] }}
      >
        {task.priority}
      </span>

      <h4 style={{
        textDecoration: status === 'completed' ? 'line-through' : 'none'
      }}>
        {task.title}
      </h4>

      {task.description && (
        <p className="description">{task.description}</p>
      )}

      <span className="date">
        {new Date(task.createdAt).toLocaleDateString()}
      </span>
    </div>

    <div className="task-actions">
      <select
        value={status}
        onChange={(e) => handleStatusChange(e.target.value)}
      >
        <option value="pending">Pending</option>
        <option value="in-progress">In Progress</option>
        <option value="completed">Completed</option>
      </select>

      <button onClick={handleDelete} className="delete-btn">
        🗑️
      </button>
    </div>
  </div>
);
}

```

## 19. src/components/TaskList.jsx

```

import TaskItem from './TaskItem';

export default function TaskList({ tasks, onUpdate, onDelete }) {
  if (tasks.length === 0) {
    return <p className="no-tasks">No tasks yet. Add one above!</p>;
  }
}

```

```

    }

    return (
      <div className="task-list">
        {tasks.map(task => (
          <TaskItem
            key={task._id}
            task={task}
            onUpdate={onUpdate}
            onDelete={onDelete}
          />
        ))}
      </div>
    );
  }
}

```

## 20. src/pages/Login.jsx

```

import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export default function Login() {
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const { login } = useAuth();
  const navigate = useNavigate();

  const handleSubmit = async (e) => {
    e.preventDefault();

    try {
      setError('');
      setLoading(true);
      await login(email, password);
      navigate('/dashboard');
    } catch (err) {
      setError(err.message);
    } finally {
      setLoading(false);
    }
  };

  return (
    <div className="auth-page">
      <form onSubmit={handleSubmit} className="auth-form">
        <h2>Login</h2>

```

```

      {error && <div className="error">{error}</div>}

      <div className="form-group">
        <label>Email</label>
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
        />
      </div>

      <div className="form-group">
        <label>Password</label>
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          required
        />
      </div>

      <button type="submit" disabled={loading}>
        {loading ? 'Logging in...' : 'Login'}
      </button>

      <p style={{ marginTop: '1rem', textAlign: 'center' }}>
        Don't have an account? <Link to="/register">Register</Link>
      </p>
    </form>
  </div>
);
}

```

## 21. src/pages/Register.jsx

```

import { useState } from 'react';
import { Link, useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export default function Register() {
  const [name, setName] = useState('');
  const [email, setEmail] = useState('');
  const [password, setPassword] = useState('');
  const [error, setError] = useState('');
  const [loading, setLoading] = useState(false);

  const { register } = useAuth();
  const navigate = useNavigate();

```

```

const handleSubmit = async (e) => {
  e.preventDefault();

  try {
    setError('');
    setLoading(true);
    await register(name, email, password);
    navigate('/dashboard');
  } catch (err) {
    setError(err.message);
  } finally {
    setLoading(false);
  }
};

return (
  <div className="auth-page">
    <form onSubmit={handleSubmit} className="auth-form">
      <h2>Register</h2>

      {error && <div className="error">{error}</div>}

      <div className="form-group">
        <label>Name</label>
        <input
          type="text"
          value={name}
          onChange={(e) => setName(e.target.value)}
          required
        />
      </div>

      <div className="form-group">
        <label>Email</label>
        <input
          type="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
        />
      </div>

      <div className="form-group">
        <label>Password</label>
        <input
          type="password"
          value={password}
          onChange={(e) => setPassword(e.target.value)}
          minLength={6}
          required
        />
      </div>
    </form>
  </div>
);

```

```

    </div>

    <button type="submit" disabled={loading}>
      {loading ? 'Registering...' : 'Register'}
    </button>

    <p style={{ marginTop: '1rem', textAlign: 'center' }}>
      Already have an account? <Link to="/login">Login</Link>
    </p>
  </form>
</div>
);
}

```

## 22. src/pages/Dashboard.jsx

```

import { useState, useEffect } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';
import TaskForm from '../components/TaskForm';
import TaskList from '../components/TaskList';

export default function Dashboard() {
  const [tasks, setTasks] = useState([]);
  const [loading, setLoading] = useState(true);
  const [error, setError] = useState('');
  const [filter, setFilter] = useState('all');

  const { user, isAuthenticated } = useAuth();
  const navigate = useNavigate();

  // Redirect if not logged in
  useEffect(() => {
    if (!isAuthenticated) {
      navigate('/login');
    }
  }, [isAuthenticated, navigate]);

  // Fetch tasks
  useEffect(() => {
    const fetchTasks = async () => {
      try {
        const response = await fetch('http://localhost:5000/api/tasks', {
          headers: {
            'Authorization': `Bearer ${user?.token}`
          }
        });

        const data = await response.json();

```

```

        if (response.ok) {
            setTasks(data);
        } else {
            setError(data.message);
        }
    } catch (err) {
        setError('Failed to fetch tasks');
    } finally {
        setLoading(false);
    }
};

if (user) {
    fetchTasks();
}, [user]);

// Task handlers
const handleTaskAdded = (newTask) => {
    setTasks(prev => [newTask, ...prev]);
};

const handleTaskUpdate = (updatedTask) => {
    setTasks(prev => prev.map(t =>
        t._id === updatedTask._id ? updatedTask : t
    ));
};

const handleTaskDelete = (taskId) => {
    setTasks(prev => prev.filter(t => t._id !== taskId));
};

// Filter tasks
const filteredTasks = tasks.filter(task => {
    if (filter === 'all') return true;
    return task.status === filter;
});

// Stats
const stats = {
    total: tasks.length,
    pending: tasks.filter(t => t.status === 'pending').length,
    inProgress: tasks.filter(t => t.status === 'in-progress').length,
    completed: tasks.filter(t => t.status === 'completed').length
};

if (loading) return <div className="loading">Loading...</div>;

return (
    <div className="dashboard">
        <h1>My Tasks</h1>

```

```

{/* Stats */}
<div className="stats">
  <div className="stat">
    <span className="stat-number">{stats.total}</span>
    <span className="stat-label">Total</span>
  </div>
  <div className="stat pending">
    <span className="stat-number">{stats.pending}</span>
    <span className="stat-label">Pending</span>
  </div>
  <div className="stat in-progress">
    <span className="stat-number">{stats.inProgress}</span>
    <span className="stat-label">In Progress</span>
  </div>
  <div className="stat completed">
    <span className="stat-number">{stats.completed}</span>
    <span className="stat-label">Completed</span>
  </div>
</div>

{/* Add Task Form */}
<TaskForm onTaskAdded={handleTaskAdded} />

{/* Filter */}
<div className="filter-bar">
  <button
    className={filter === 'all' ? 'active' : ''}
    onClick={() => setFilter('all')}
  >
    All
  </button>
  <button
    className={filter === 'pending' ? 'active' : ''}
    onClick={() => setFilter('pending')}
  >
    Pending
  </button>
  <button
    className={filter === 'in-progress' ? 'active' : ''}
    onClick={() => setFilter('in-progress')}
  >
    In Progress
  </button>
  <button
    className={filter === 'completed' ? 'active' : ''}
    onClick={() => setFilter('completed')}
  >
    Completed
  </button>
</div>

{/* Task List */}

```

```

    {error && <div className="error">{error}</div>}

    <TaskList
      tasks={filteredTasks}
      onUpdate={handleTaskUpdate}
      onDelete={handleTaskDelete}
    />
  </div>
);
}

```

## 23. src/App.jsx

```

import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from './context/AuthContext';
import Header from './components/Header';
import Home from './pages/Home';
import Login from './pages/Login';
import Register from './pages/Register';
import Dashboard from './pages/Dashboard';

// Protected Route Component
function ProtectedRoute({ children }) {
  const { isAuthenticated, loading } = useAuth();

  if (loading) return <div>Loading...</div>;

  return isAuthenticated ? children : <Navigate to="/login" />;
}

function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Header />
        <main className="container">
          <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/login" element={<Login />} />
            <Route path="/register" element={<Register />} />
            <Route
              path="/dashboard"
              element={
                <ProtectedRoute>
                  <Dashboard />
                </ProtectedRoute>
              }
            />
          </Routes>
        </main>
      </BrowserRouter>
    </AuthProvider>
  );
}

```

```

        </BrowserRouter>
      </AuthProvider>
    );
  }

  export default App;

```

## 24. src/pages/Home.jsx

```

import { Link } from 'react-router-dom';
import { useAuth } from '../context/AuthContext';

export default function Home() {
  const { isAuthenticated } = useAuth();

  return (
    <div className="home">
      <h1>📅 Task Manager</h1>
      <p>Organize your tasks efficiently with our MERN stack app!</p>

      <div className="features">
        <div className="feature">✅ Create & manage tasks</div>
        <div className="feature">🔔 Set priorities</div>
        <div className="feature">📊 Track progress</div>
        <div className="feature">🔒 Secure authentication</div>
      </div>

      {isAuthenticated ? (
        <Link to="/dashboard" className="cta-button">
          Go to Dashboard
        </Link>
      ) : (
        <div className="cta-buttons">
          <Link to="/register" className="cta-button">
            Get Started
          </Link>
          <Link to="/login" className="cta-button secondary">
            Login
          </Link>
        </div>
      )}
    </div>
  );
}

```

## 25. src/index.css - Basic Styles

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
  background: #f5f5f5;
  color: #333;
  line-height: 1.6;
}

.container {
  max-width: 1200px;
  margin: 0 auto;
  padding: 2rem;
}

/* Header */
.header {
  background: #1a1a2e;
  color: white;
  padding: 1rem 2rem;
}

.header .container {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 0;
}

.logo {
  color: white;
  text-decoration: none;
  font-size: 1.5rem;
  font-weight: bold;
}

.header nav {
  display: flex;
  gap: 1rem;
  align-items: center;
}

.header nav a, .header nav button {
  color: white;
  text-decoration: none;
  padding: 0.5rem 1rem;
  border-radius: 4px;
}
```

```
    background: transparent;
    border: 1px solid white;
    cursor: pointer;
}

.header nav a:hover, .header nav button:hover {
    background: white;
    color: #1a1a2e;
}

/* Forms */
.auth-page {
    display: flex;
    justify-content: center;
    align-items: center;
    min-height: 60vh;
}

.auth-form, .task-form {
    background: white;
    padding: 2rem;
    border-radius: 8px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.1);
    width: 100%;
    max-width: 400px;
}

.form-group {
    margin-bottom: 1rem;
}

.form-group label {
    display: block;
    margin-bottom: 0.5rem;
    font-weight: 500;
}

.form-group input, .form-group textarea, .form-group select {
    width: 100%;
    padding: 0.75rem;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 1rem;
}

button {
    width: 100%;
    padding: 0.75rem;
    background: #4f46e5;
    color: white;
    border: none;
    border-radius: 4px;
}
```

```

    font-size: 1rem;
    cursor: pointer;
}

button:hover {
    background: #4338ca;
}

button:disabled {
    opacity: 0.5;
    cursor: not-allowed;
}

.error {
    background: #fee2e2;
    color: #dc2626;
    padding: 0.75rem;
    border-radius: 4px;
    margin-bottom: 1rem;
}

/* Dashboard */
.dashboard h1 {
    margin-bottom: 2rem;
}

.stats {
    display: grid;
    grid-template-columns: repeat(4, 1fr);
    gap: 1rem;
    margin-bottom: 2rem;
}

.stat {
    background: white;
    padding: 1.5rem;
    border-radius: 8px;
    text-align: center;
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

.stat-number {
    display: block;
    font-size: 2rem;
    font-weight: bold;
    color: #4f46e5;
}

.stat.pending .stat-number { color: #f59e0b; }
.stat.in-progress .stat-number { color: #3b82f6; }
.stat.completed .stat-number { color: #10b981; }

```

```
/* Task Form */
.task-form {
  margin-bottom: 2rem;
  max-width: 100%;
}

.task-form h3 {
  margin-bottom: 1rem;
}

/* Filter Bar */
.filter-bar {
  display: flex;
  gap: 0.5rem;
  margin-bottom: 1rem;
}

.filter-bar button {
  width: auto;
  padding: 0.5rem 1rem;
  background: #e5e7eb;
  color: #333;
}

.filter-bar button.active {
  background: #4f46e5;
  color: white;
}

/* Task List */
.task-list {
  display: flex;
  flex-direction: column;
  gap: 1rem;
}

.task-item {
  background: white;
  padding: 1rem;
  border-radius: 8px;
  display: flex;
  justify-content: space-between;
  align-items: flex-start;
  box-shadow: 0 2px 5px rgba(0,0,0,0.1);
  border-left: 4px solid #4f46e5;
}

.task-item.completed {
  border-left-color: #10b981;
  opacity: 0.7;
}
```

```
.task-item h4 {
  margin: 0.5rem 0;
}

.priority-badge {
  display: inline-block;
  padding: 0.25rem 0.5rem;
  border-radius: 4px;
  color: white;
  font-size: 0.75rem;
  text-transform: uppercase;
}

.task-actions {
  display: flex;
  gap: 0.5rem;
  align-items: center;
}

.task-actions select {
  padding: 0.5rem;
}

.delete-btn {
  width: auto;
  padding: 0.5rem;
  background: #ef4444;
}

/* Home */
.home {
  text-align: center;
  padding: 4rem 2rem;
}

.home h1 {
  font-size: 3rem;
  margin-bottom: 1rem;
}

.features {
  display: flex;
  justify-content: center;
  gap: 2rem;
  margin: 2rem 0;
  flex-wrap: wrap;
}

.feature {
  background: white;
  padding: 1rem 2rem;
  border-radius: 8px;
}
```

```
    box-shadow: 0 2px 5px rgba(0,0,0,0.1);
}

.cta-buttons {
  display: flex;
  gap: 1rem;
  justify-content: center;
  margin-top: 2rem;
}

.cta-button {
  display: inline-block;
  padding: 1rem 2rem;
  background: #4f46e5;
  color: white;
  text-decoration: none;
  border-radius: 8px;
  font-weight: bold;
}

.cta-button.secondary {
  background: transparent;
  border: 2px solid #4f46e5;
  color: #4f46e5;
}

.no-tasks {
  text-align: center;
  color: #666;
  padding: 2rem;
}

.loading {
  text-align: center;
  padding: 2rem;
}
```

---

## How to Run

### Backend:

```
cd backend
npm install
npm run dev
```

### Frontend:

```
cd frontend
npm install
```

```
npm run dev
```

## API Endpoints Summary

| Method | Endpoint           | Description        | Auth |
|--------|--------------------|--------------------|------|
| POST   | /api/auth/register | Register user      | No   |
| POST   | /api/auth/login    | Login user         | No   |
| GET    | /api/auth/me       | Get current user   | Yes  |
| GET    | /api/tasks         | Get all user tasks | Yes  |
| GET    | /api/tasks/:id     | Get single task    | Yes  |
| POST   | /api/tasks         | Create task        | Yes  |
| PUT    | /api/tasks/:id     | Update task        | Yes  |
| DELETE | /api/tasks/:id     | Delete task        | Yes  |

## Key Concepts Demonstrated

### Backend

- ✓ Express.js routing & middleware
- ✓ MongoDB with Mongoose
- ✓ User authentication with JWT
- ✓ Password hashing with bcrypt
- ✓ Protected routes
- ✓ Error handling
- ✓ CRUD operations

### Frontend

- ✓ React functional components
- ✓ React hooks (useState, useEffect, useContext)
- ✓ Context API for global state
- ✓ Custom hooks (useFetch)
- ✓ React Router for navigation
- ✓ Protected routes
- ✓ Form handling
- ✓ API integration

**This is a complete, working MERN application!** 🌈